SEMANA 13 · SESIÓN 13

Procesamiento de formularios, manejo de errores y validación de datos

Angular 22 (Reactive Forms) + **Spring Boot 4** (Bean Validation)

Desarrollo Web Integrado (100000ST61)
Unidad 3: Integración de proyectos web (back-end y front-end)

Docente: Mag. Ing. Marco Antonio Flores Torres
Universidad Tecnológica del Perú · Lima Sur



INICIO Agenda

UTP

Ruta de la sesión de hoy (45 minutos)

Inicio
8 min
Rompehielo, repaso de la sesión 12 y saberes previos

→

Utilidad
6 min
Preguntas retadoras, logro y para qué sirve el tema

→

Transformación
18 min
Reactive Forms, validadores, Bean Validation y manejo de errores

→

Práctica
9 min
Tres ejercicios: inicial, intermedio y avanzado

→

Cierre
4 min
Conclusiones, evaluación y mensaje final

Caso de estudio de toda la sesión: el formulario de registro de clientes de TechStore Perú SAC.

¡Caza los 5 errores!



Dinámica en parejas (2 minutos)

Observa el formulario de registro de la "Tienda XYZ".

Encuentra 5 problemas de validación, de mensajes o de seguridad.

Anótalos en una hoja o en el chat de la clase.

Reto

La primera pareja que encuentre los 5 errores los explica a toda la clase.

Pista: piensa como usuario (¿entiendo qué pasó?) y como atacante (¿qué información me regala?).

Los 5 errores y cómo los corregiremos hoy



1. **Muestra el error técnico (500 y traza de Java)**
   Solución: Mensaje amigable al usuario y detalle solo en el log del servidor.

2. **Campo sin etiqueta: solo tiene placeholder**
   Solución: Usar <label> visible; el placeholder desaparece al escribir.

3. **Acepta el DNI "12AB" y lo marca como correcto**
   Solución: Validar formato: exactamente 8 dígitos, en Angular y en Spring Boot.

4. **Contraseña de 3 caracteres y sin confirmar**
   Solución: Longitud mínima y validador cruzado contraseña = confirmación.

5. **Mensaje genérico "Datos inválidos"**
   Solución: Mensaje específico debajo de cada campo con error.

INICIO Repaso de la sesión 12 UTP

¿Recuerdas la clase anterior? Tres preguntas rápidas

1

¿Qué etiqueta indica en qué parte de la pantalla se dibuja el componente de la ruta activa?

Pista: va en app.html

2

¿Qué directiva usamos en el HTML para navegar entre páginas sin recargar el navegador?

Pista: reemplaza al href

3

En Angular 22, ¿con qué función recibe un componente hijo un dato que le envía su padre?

Pista: comunicación padre → hijo

¿Quedó alguna duda de rutas o de comunicación entre componentes? Este es el momento de preguntarla.

INICIO Contextualización UTP

Todos los días llenas formularios… y alguien los programó

Billeteras digitales

Yape o Plin piden un celular de 9 dígitos y un monto mayor a cero antes de enviar dinero.

SUNAT

Para emitir una factura se necesita un RUC válido de 11 dígitos.

Portal universitario

La matrícula valida cursos, cruces de horario y créditos disponibles.

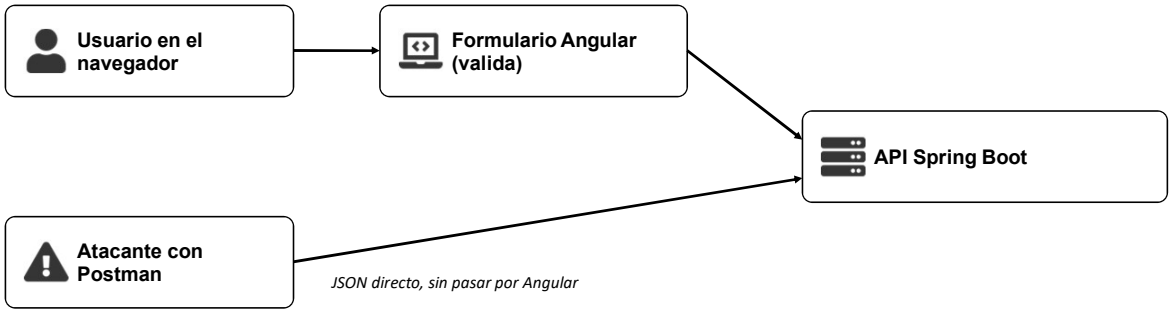
Delivery

La app no confirma el pedido sin dirección, teléfono y medio de pago válidos.

Un dato mal validado puede convertirse en un pedido perdido, un cobro duplicado o una brecha de seguridad.

INICIO Saberes previos UTP

Si Angular ya valida… ¿Spring Boot debe validar otra vez?



Votemos a mano alzada:

A) No, ya validó Angular	B) Sí, porque alguien puede saltarse el front-end	C) Solo si hay base de datos

UTILIDAD Preguntas retadoras · sesión 12 UTP

Piensa antes de responder: tres retos de rutas y servicios

1 La ruta comodín quedó primero

código

```
const routes = [
  { path: '**',
    component: NoEncontrado },
  { path: 'catalogo',
    component: Catalogo },
];
```

¿Qué verá el usuario al entrar a /catalogo? ¿Por qué?

2 El input() siempre llega vacío

código

```
// URL: /catalogo?categoria=laptops
categoria = input<string>();
// app.config.ts
provideRouter(routes)
```

¿Por qué categoria() es undefined? ¿Qué falta?

3 El carrito no se actualiza

código

```
@Component({
  selector: 'app-catalogo',
  providers: [CarritoService]
})
```

CarritoService ya tiene providedIn: 'root'. ¿Por qué el Header muestra 0 productos?

Trabajen en parejas: tienen 1 minuto para responder las tres.

UTILIDAD Logro de aprendizaje UTP

Logro de la sesión

Al finalizar la sesión, el estudiante implementa formularios reactivos en Angular con validación de datos y manejo de errores en el cliente y en el servidor (Bean Validation y @RestControllerAdvice en Spring Boot), *aplicando buenas prácticas de experiencia de usuario y seguridad en el caso TechStore Perú.*

Tiempo	Sujeto	Verbo	Contenido	Condición
Al finalizar la sesión	El estudiante	Implementa	Formularios con validación y manejo de errores	Con buenas prácticas de UX y seguridad

En tu desarrollo profesional: todo sistema empresarial (ventas, banca, salud, educación) recibe datos por formularios. Validarlos bien es una competencia que te piden desde el primer empleo.

UTILIDAD ¿Para qué me sirve? UTP

Completa la frase: “Validar datos me sirve para…”

Calidad de datos

La base de datos guarda solo información correcta: DNI reales, correos con formato válido y montos positivos. Los reportes y la facturación salen bien.

Seguridad

El back-end rechaza datos manipulados que llegan sin pasar por la pantalla y no revela detalles técnicos que un atacante podría aprovechar.

Experiencia de usuario

El usuario sabe exactamente qué campo corregir y por qué. Menos frustración, menos abandono del registro y más ventas.

En tu carrera

Lo usarás en tu proyecto integrador (APF3 en la semana 15), en prácticas preprofesionales y en entrevistas técnicas, donde es común pedir: “valida este formulario y maneja los errores de la API”.

UTILIDAD Ejemplos reales

UTP

Reglas de validación del mundo real que aplicaremos

Sistema o servicio	Dato	Regla de validación	¿Qué problema evita?
Reniec / registro de clientes	DNI	Exactamente 8 dígitos	Clientes duplicados o inexistentes
SUNAT / facturación	RUC	11 dígitos, empieza con 10 o 20	Facturas rechazadas por la SUNAT
Billeteras digitales	Celular	9 dígitos, empieza con 9	Dinero enviado a un número equivocado
Bancos / transferencias	CCI	Exactamente 20 dígitos	Transferencias fallidas
Tiendas online	Correo	Formato usuario@dominio	Comprobantes que nunca llegan
Aerolíneas	Fecha de viaje	Debe ser una fecha futura	Reservas imposibles

Las tres primeras reglas (DNI, RUC y celular) las programarás hoy en Angular y en Spring Boot.

TRANSFORMACIÓN

Validación en dos capas: Angular + Spring Boot

Parte A

Formularios reactivos en Angular

Parte B

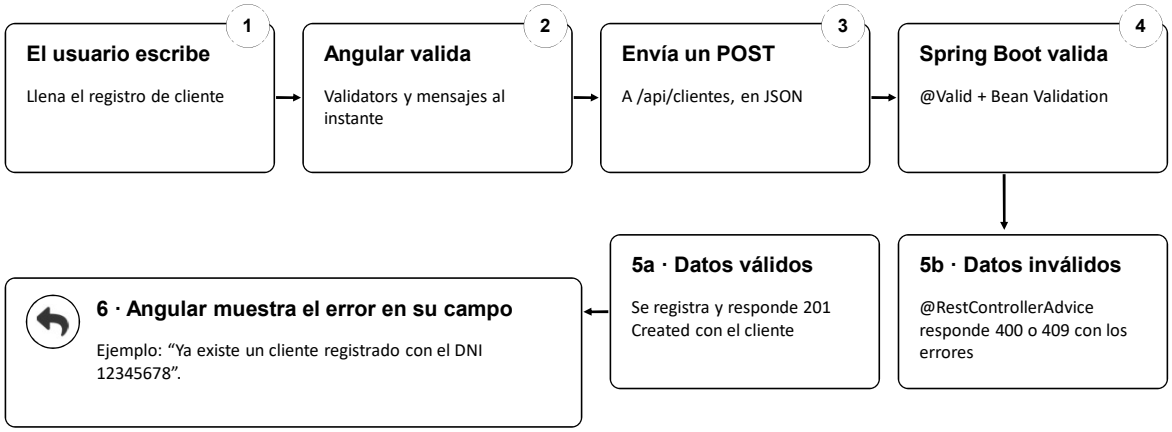
Bean Validation y errores en Spring Boot

Parte C

Integración de errores front-back

TRANSFORMACIÓN Visión general UTP

El viaje de un dato: del formulario a la base de datos



El front-end valida para AYUDAR al usuario; el back-end valida para PROTEGER el sistema.

TRANSFORMACIÓN Parte A · Angular UTP

Tres formas de crear formularios en Angular

Característica	Template-driven	Reactive Forms	Signal Forms (Angular 22)
Se importa	FormsModule	ReactiveFormsModule	FormField (@angular/forms/signals)
¿Dónde vive el modelo?	En la plantilla, con [(ngModel)]	En la clase: FormGroup y FormControl	En un signal, con la función form()
Validación	Atributos HTML: required, minlength	Validators y funciones propias	Reglas: required(), email(), pattern()
Ideal para	Formularios pequeños y simples	Formularios grandes y empresariales	Proyectos nuevos con signals
Madurez	Muy estable	Muy estable, la más usada	Estable desde junio de 2026
En esta sesión	Solo como referencia	**Tema principal**	Vista previa

Las tres formas conviven y usan los mismos conceptos: valor, validez, errores y si el campo fue tocado.

TRANSFORMACIÓN Parte A · Reactive Forms UTP

Las piezas de un formulario reactivo

FormControl
Un campo: guarda su valor, su validez y sus errores.

FormGroup
Agrupa varios controles: el formulario completo.

FormArray
Lista dinámica de controles (ej.: varios teléfonos).

NonNullableFormBuilder
Crea el formulario con menos código y sin valores null.

Validators
Reglas listas: required, email, minLength, pattern…

Analogía: el FormGroup es la ficha de inscripción completa; cada FormControl es una casilla de la ficha.

Conexión en la plantilla HTML

```
<form [formGroup]="form"
      (ngSubmit)="registrar()">
  <input formControlName="dni" />
</form>
```

Directiva	Función
[formGroup]	Une el <form> con el FormGroup de la clase
formControlName	Une un <input> con un control del grupo
(ngSubmit)	Ejecuta un método al enviar el formulario

TRANSFORMACIÓN Parte A · Estados UTP

Cada campo sabe en qué estado está

`valid / invalid`	¿Cumple todas sus reglas?
`touched / untouched`	¿El usuario entró y salió del campo?
`dirty / pristine`	¿El usuario cambió el valor?
`errors`	Objeto con los errores: { required: true }
`pending`	Validación asíncrona en curso (ej.: consultar a la API)

Clases CSS automáticas: ng-valid, ng-invalid, ng-touched, ng-dirty



Panel didáctico del taller: muestra el estado del formulario en tiempo real.

Regla de oro para mostrar errores

```
control.invalid && control.touched
```

TRANSFORMACIÓN Ejemplo 1 · Paso 1 UTP

Ejemplo 1: crear el formulario en la clase TypeScript

pages/registro-cliente/registro-cliente.ts

```ts
import { Component, inject, signal } from '@angular/core';
import { NonNullableFormBuilder, ReactiveFormsModule,
         Validators } from '@angular/forms';

@Component({
  selector: 'app-registro-cliente',
  imports: [ReactiveFormsModule],
  templateUrl: './registro-cliente.html',
})
export class RegistroCliente {
  private readonly fb = inject(NonNullableFormBuilder);

  protected readonly form = this.fb.group({
    nombres: ['', [Validators.required, Validators.minLength(2)]],
    dni:     ['', [Validators.required, dniValidator]],
    email:   ['', [Validators.required, Validators.email]],
    celular: ['', [Validators.required, celularPeruValidator]],
    password: ['', [Validators.required, Validators.minLength(8)]],
    confirmarPassword: ['', Validators.required],
    aceptaTerminos: [false, Validators.requiredTrue],
  }, {
    validators: camposIgualesValidator('password', 'confirmarPassword'),
  });
}
```

ReactiveFormsModule: se importa en el componente standalone.

fb.group({...}): cada propiedad es un control.

['', [...]]: valor inicial y lista de validadores.

requiredTrue: obliga a marcar el checkbox.

{ validators }: reglas que miran varios campos a la vez.

TRANSFORMACIÓN Ejemplo 1 · Paso 2 UTP

Ejemplo 1: conectar la plantilla y mostrar errores

registro-cliente.html

```html
<form [formGroup]="form" (ngSubmit)="registrar()" novalidate>
  <label for="dni">DNI *</label>
  <input id="dni" formControlName="dni" maxlength="8"
         [class.invalido]="mostrarError('dni')" />
  @if (mostrarError('dni')) {
    <small class="error">{{ mensajeError('dni') }}</small>
  }
  <button type="submit">Registrar</button>
</form>
```

registro-cliente.ts

```ts
protected registrar(): void {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  // ... enviar al back-end
}
```



Resultado al presionar Registrar con el formulario vacío

TRANSFORMACIÓN Parte A · Validators UTP

Validadores integrados de Angular

Validador	Clave del error	Ejemplo en TechStore
Validators.required	required	Nombres, DNI y correo son obligatorios
Validators.requiredTrue	required	Aceptar los términos y condiciones
Validators.email	email	ana@correo.pe es válido; ana@ no lo es
Validators.minLength(8)	minlength	La contraseña tiene al menos 8 caracteres
Validators.maxLength(50)	maxlength	Los nombres no superan 50 caracteres
Validators.min(1) / max(100)	min / max	Precio mayor o igual a 1; descuento hasta 100
Validators.pattern(/^\d{8}$/)	pattern	El DNI tiene exactamente 8 dígitos

Leer un error en TypeScript

```
this.form.get('email')?.hasError('email')   // true si el formato del correo es incorrecto
```


TRANSFORMACIÓN Ejemplo 2 UTP

Ejemplo 2: validadores personalizados para datos peruanos

validators/peru-validators.ts

```
import { AbstractControl, ValidationErrors,
         ValidatorFn } from '@angular/forms';

// DNI: exactamente 8 dígitos
export function dniValidator(control: AbstractControl)
    : ValidationErrors | null {
  const valor = control.value as string;
  if (!valor) return null;   // el vacío lo valida required
  return /^\d{8}$/.test(valor) ? null : { dniInvalido: true };
}

// Validador cruzado: dos campos deben ser iguales
export function camposIgualesValidator(campo: string,
    confirmacion: string): ValidatorFn {
  return (grupo: AbstractControl) => {
    const v1 = grupo.get(campo)?.value;
    const v2 = grupo.get(confirmacion)?.value;
    if (!v2) return null;
    return v1 === v2 ? null : { passwordsNoCoinciden: true };
  };
}
```

Devuelve null

El valor es correcto: el control queda válido.

Devuelve { dniInvalido: true }

El valor es incorrecto: esa clave aparece en control.errors.

Mismo patrón

celularPeruValidator: /^9\d{8}$/
rucValidator: /^(10|20)\d{9}$/

TRANSFORMACIÓN Ejemplo 2 · Continuación UTP

Validación condicional y mensajes claros

Validación condicional del RUC

```
constructor() {
  this.form.controls.tipoComprobante.valueChanges
    .pipe(takeUntilDestroyed())
    .subscribe((tipo) => {
      const ruc = this.form.controls.ruc;
      if (tipo === 'FACTURA') {
        ruc.setValidators([Validators.required,
                           rucValidator]);
      } else {
        ruc.clearValidators();
        ruc.setValue('');
      }
      ruc.updateValueAndValidity();
    });
}
```

Un mensaje claro por cada error

```
protected mensajeError(campo: string): string {
  const e = this.form.get(campo)?.errors;
  if (!e) return '';
  if (e['servidor']) return e['servidor'];
  if (e['required'])
    return 'Este campo es obligatorio.';
  if (e['minlength']) {
    const n = e['minlength'].requiredLength;
    return `Debe tener al menos ${n} caracteres.`;
  }
  if (e['email']) return 'Ingresa un correo válido.';
  if (e['dniInvalido'])
    return 'El DNI debe tener exactamente 8 dígitos.';
  return 'El valor ingresado no es válido.';
}
```

Buenas prácticas: mensaje específico, cerca del campo, en lenguaje del usuario y que diga cómo corregir.

TRANSFORMACIÓN Resultado UTP

Así se ve el formulario de TechStore con las validaciones



Errores de formato, RUC condicional y contraseñas distintas



Registro exitoso (201 Created)

Lo que ya logramos

- Errores visibles solo al tocar el campo
- Mensajes específicos por campo
- Reglas peruanas: DNI, celular y RUC
- Validación cruzada de contraseñas

TRANSFORMACIÓN Vista previa UTP

Vista previa: Signal Forms, estable desde Angular 22

suscripcion.ts (Signal Forms)

```ts
import { Component, signal } from '@angular/core';
import { form, FormField, required, email, pattern }
  from '@angular/forms/signals';

@Component({
  selector: 'app-suscripcion',
  imports: [FormField],
  template: `
    <input [formField]="suscripcion.email" />
    @if (suscripcion.email().touched()
         && suscripcion.email().invalid()) {
      <small>Revisa tu correo</small>
    }`,
})
export class Suscripcion {
  modelo = signal({ email: '', celular: '' });
  suscripcion = form(this.modelo, (campo) => {
    required(campo.email, { message: 'Obligatorio' });
    email(campo.email, { message: 'Correo no válido' });
    pattern(campo.celular, /^9\d{8}$/);
  });
}
```

Reactive Forms	Signal Forms
fb.group({ ... })	form(signal, reglas)
formControlName="email"	[formField]="f.email"
Validators.email	email(campo.email)
control.invalid	f.email().invalid()

Mismos conceptos, nueva sintaxis. Si dominas Reactive Forms, migrar será sencillo.

TRANSFORMACIÓN Parte B · Spring Boot UTP

Nunca confíes en el cliente: Bean Validation

Anotación	¿Qué valida?	Ejemplo
@NotBlank	Texto no nulo, no vacío y no solo espacios	nombres, dni
@Size(min, max)	Longitud de un texto o lista	@Size(min = 2, max = 50)
@Pattern(regexp)	Formato con expresión regular	@Pattern(regexp = "\\d{8}")
@Email	Formato de correo	email
@NotNull	El valor no puede ser null	aceptaTerminos
@Positive / @PositiveOrZero	Número mayor que 0 / mayor o igual a 0	precio / stock
@Min / @Max	Rango numérico	@Min(18) edad
@AssertTrue	Debe ser true (sirve para reglas cruzadas)	isRucValido()
@Past / @Future	Fecha pasada / futura	fechaNacimiento / fechaViaje

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-validation
  </artifactId>
</dependency>
```

¿Por qué en el DTO?

El DTO (record) describe exactamente qué datos acepta la API. Las reglas viajan junto con los datos.

TRANSFORMACIÓN Ejemplo 3 · Paso 1 UTP

Ejemplo 3: el DTO con reglas y @Valid en el controlador

dto/ClienteRequest.java

```java
public record ClienteRequest(
    @NotBlank(message = "Los nombres son obligatorios")
    @Size(min = 2, max = 50,
          message = "Entre 2 y 50 caracteres")
    String nombres,

    @NotBlank(message = "El DNI es obligatorio")
    @Pattern(regexp = "\\d{8}",
          message = "El DNI debe tener exactamente 8 dígitos")
    String dni,

    @NotBlank @Email(message = "Correo con formato inválido")
    String email,
    String tipoComprobante,
    String ruc
    // ... celular, password, aceptaTerminos
) {
  @AssertTrue(message = "Para FACTURA el RUC es obligatorio")
  public boolean isRucValido() {
    if (!"FACTURA".equals(tipoComprobante)) return true;
    return ruc != null && ruc.matches("(10|20)\\d{9}");
  }
}
```

controller/ClienteController.java

```java
@RestController
@RequestMapping("/api/clientes")
@CrossOrigin(origins = "http://localhost:4200")
public class ClienteController {
  // ... constructor con ClienteService

  @PostMapping
  public ResponseEntity<ClienteResponse>
      registrar(@Valid @RequestBody
                ClienteRequest request) {
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(service.registrar(request));
  }
}
```

⚠ Sin @Valid las anotaciones del DTO NO se ejecutan: es el error más común.

TRANSFORMACIÓN Ejemplo 3 · Paso 2 UTP

Ejemplo 3: un solo lugar para manejar los errores

exception/GlobalExceptionHandler.java

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<ErrorResponse> manejarValidacion(
      MethodArgumentNotValidException ex, HttpServletRequest req) {
    Map<String, String> errores = new LinkedHashMap<>();
    for (FieldError e : ex.getBindingResult().getFieldErrors()) {
      errores.putIfAbsent(e.getField(), e.getDefaultMessage());
    }
    return construir(HttpStatus.BAD_REQUEST,
        "Hay campos con datos inválidos", req, errores);
  }

  @ExceptionHandler(DniDuplicadoException.class)
  public ResponseEntity<ErrorResponse> manejarDniDuplicado(
      DniDuplicadoException ex, HttpServletRequest req) {
    return construir(HttpStatus.CONFLICT,
        "No se pudo completar el registro.", req,
        Map.of("dni", ex.getMessage()));
  }
  // + 404 (no encontrado) y 500 (error no previsto)
}
```

Excepción → respuesta

`MethodArgumentNotValidException` → **400 Bad Request**

`HttpMessageNotReadableException` → **400 · JSON mal escrito**

`DniDuplicadoException` → **409 Conflict**

`RecursoNoEncontradoException` → **404 Not Found**

`Exception (cualquier otra)` → **500 · mensaje amigable**

TRANSFORMACIÓN Ejemplo 3 · Resultado UTP

La respuesta de error: clara y útil para Angular

POST /api/clientes → 400 Bad Request

```
{
  "timestamp": "2026-11-04T19:35:12.481",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Hay campos con datos inválidos",
  "ruta": "/api/clientes",
  "errores": {
    "dni": "El DNI debe tener exactamente 8 dígitos",
    "email": "El correo no tiene un formato válido",
    "rucValido": "Para FACTURA el RUC es obligatorio"
  }
}
```

Código	Significado	En TechStore
201	Created	Cliente registrado
400	Bad Request	DNI con letras, JSON mal escrito
404	Not Found	GET /api/clientes/999
409	Conflict	DNI ya registrado
500	Internal Server Error	Error no previsto (sin traza)

La clave de cada error (dni, email…) coincide con el formControlName de Angular. ¡Ese es el puente entre las dos capas!

TRANSFORMACIÓN Parte C · Integración UTP

Parte C: mostrar en el formulario los errores del servidor

registro-cliente.ts

```
this.clienteService.registrar(datos).subscribe({
  next: (cliente) => this.clienteRegistrado.set(cliente),
  error: (err: HttpErrorResponse) =>
    this.mostrarErroresDelServidor(err),
});

private mostrarErroresDelServidor(err: HttpErrorResponse) {
  if (err.status === 0) {
    this.mensajeGeneral.set('No se pudo conectar con el servidor.');
    return;
  }
  const errores = err.error?.errores ?? {};
  for (const [campo, mensaje] of Object.entries(errores)) {
    const control = this.form.get(campo);
    if (control) {
      control.setErrors({ servidor: mensaje });
      control.markAsTouched();
    }
  }
  this.mensajeGeneral.set(err.error?.mensaje);
}
```



Respuesta 409 del back-end mostrada bajo el campo DNI

PRÁCTICA Ejercicios

UTP

Practica: tres retos de menor a mayor dificultad

INICIAL

Ejercicio 1: Suscripción al boletín

- Formulario con nombre (obligatorio, mínimo 3 letras) y correo (obligatorio, formato válido).
- Botón deshabilitado mientras el formulario sea inválido.
- Al enviar, mostrar "¡Gracias, {nombre}!" con un signal y limpiar el formulario.

INTERMEDIO

Ejercicio 2: Nuevo producto (panel admin)

- Campos: nombre, categoría (select), precio ≥ 1, stock entero ≥ 0 y precio de oferta opcional.
- Validador de grupo: la oferta debe ser menor que el precio.
- Al guardar, agregar el producto a una lista con signal.update().

AVANZADO

Ejercicio 3: API de productos con errores

- ProductoRequest con @NotBlank, @Pattern("TS-0001"), @Positive y @PositiveOrZero.
- POST /api/productos responde 201; si el código ya existe, 409 desde el @RestControllerAdvice.
- Probar 3 casos en Postman: válido, inválido y duplicado.

PRÁCTICA Solución · Ejercicio 1

UTP

Solución del ejercicio 1: suscripción al boletín

boletin.ts

```
@Component({
  selector: 'app-boletin',
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="suscribir()">
      <input formControlName="nombre" placeholder="Nombre" />
      @if (form.controls.nombre.touched && form.controls.nombre.invalid) {
        <small>Nombre obligatorio (mín. 3 letras)</small>
      }
      <input formControlName="email" placeholder="Correo" />
      <button [disabled]="form.invalid">Suscribirme</button>
    </form>
    @if (mensaje()) { <p>{{ mensaje() }}</p> }`,
})
export class Boletin {
  private readonly fb = inject(NonNullableFormBuilder);
  protected readonly mensaje = signal('');
  protected readonly form = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3)]],
    email: ['', [Validators.required, Validators.email]],
  });
  protected suscribir(): void {
    this.mensaje.set(`¡Gracias, ${this.form.value.nombre}!`);
    this.form.reset();
  }
}
```

Paso 1: importar ReactiveFormsModule.

Paso 2: crear el grupo con dos controles y sus validadores.

Paso 3: conectar [formGroup] y formControlName.

Paso 4: mostrar el error con @if (inválido y tocado).

Paso 5: deshabilitar el botón con [disabled].

Paso 6: mostrar el saludo con un signal y hacer reset().

PRÁCTICA Solución · Ejercicio 2 UTP

Solución del ejercicio 2: nuevo producto

nuevo-producto.ts (lógica)

```
function ofertaMenorQuePrecio(g: AbstractControl): ValidationErrors | null {
  const precio = g.get('precio')?.value;
  const oferta = g.get('precioOferta')?.value;
  if (oferta == null) return null;      // la oferta es opcional
  return oferta < precio ? null : { ofertaInvalida: true };
}

export class NuevoProducto {
  private readonly fb = inject(NonNullableFormBuilder);
  protected readonly productos = signal<Producto[]>([]);
  protected readonly form = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3)]],
    categoria: ['', Validators.required],
    precio: [0, [Validators.required, Validators.min(1)]],
    stock: [0, [Validators.required, Validators.min(0),
                Validators.pattern(/^\d+$/)]],
    precioOferta: this.fb.control<number | null>(null),
  }, { validators: ofertaMenorQuePrecio });

  protected guardar(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.productos.update((lista) => [...lista, this.form.getRawValue()]);
    this.form.reset();
  }
}
```

nuevo-producto (plantilla)

```
@if (form.hasError('ofertaInvalida')) {
  <small>
    La oferta debe ser
    menor que el precio
  </small>
}
```

El error de grupo se lee en el FormGroup (form.hasError), no en un control.

PRÁCTICA Solución · Ejercicio 3 UTP

Solución del ejercicio 3: API de productos

ProductoRequest.java y ProductoController.java

```
public record ProductoRequest(
    @NotBlank(message = "El código es obligatorio")
    @Pattern(regexp = "TS-\\d{4}",
             message = "Formato de código: TS-0001")
    String codigo,
    @NotBlank(message = "El nombre es obligatorio")
    String nombre,
    @NotNull @Positive(message = "El precio debe ser mayor que 0")
    BigDecimal precio,
    @NotNull @PositiveOrZero(message = "Stock no negativo")
    Integer stock) { }

// ProductoController
@PostMapping
public ResponseEntity<ProductoRequest> crear(
    @Valid @RequestBody ProductoRequest p) {
  if (!codigos.add(p.codigo())) {      // Set<String>
    throw new CodigoDuplicadoException(p.codigo());
  }
  return ResponseEntity.status(HttpStatus.CREATED).body(p);
}
```

GlobalExceptionHandler.java

```
@ExceptionHandler(CodigoDuplicadoException.class)
public ResponseEntity<ErrorResponse> manejarCodigo(
    CodigoDuplicadoException ex,
    HttpServletRequest req) {
  return construir(HttpStatus.CONFLICT,
      "No se pudo registrar el producto", req,
      Map.of("codigo", ex.getMessage()));
}
```

Prueba en Postman	Esperado
Datos correctos	201 Created
precio: -5, codigo: "ABC"	400 con 2 errores
Mismo código dos veces	409 Conflict

CIERRE Construimos las conclusiones UTP

Dinámica 3-2-1: ustedes construyen las conclusiones

3	**cosas que aprendí hoy**
2	**cosas que aplicaré en mi proyecto integrador**
1	**duda que todavía tengo**

Revisemos juntos cada tema y completemos la frase:

Reactive Forms sirve para…	Muestro un error cuando…	Un validador propio devuelve…
@Valid en Spring Boot…	@RestControllerAdvice permite…	Valido en dos capas porque…

CIERRE Evaluación UTP

Comprueba lo aprendido: tres preguntas

1

El usuario presiona “Registrar” sin llenar nada. ¿Qué método muestra TODOS los errores a la vez?

a) form.reset()
b) form.markAllAsTouched()
c) form.disable()
d) form.patchValue({})

2

Un compañero quitó @Valid del controlador y envía desde Postman {"dni": "12AB"}. ¿Qué pasa con @NotBlank y @Pattern del DTO?

a) Se ejecutan igual
b) No se ejecutan: Spring no valida el DTO
c) Las ejecuta Angular
d) Se convierten en un 404

3

¿Qué anotación permite manejar en UNA sola clase las excepciones de todos los controladores y responder JSON?

a) @Valid
b) @CrossOrigin
c) @RestControllerAdvice
d) @Service

CIERRE Mensaje final UTP

¡Gracias por tu participación!



"Un buen formulario no es el que nunca muestra errores, sino el que ayuda a corregirlos."

Cada validación que escribes protege a un usuario real y a la empresa que confía en tu código.



Practica

Completa el taller de laboratorio 13: formulario de TechStore y API con manejo de errores.



Participa en el foro

Comparte una captura de tus errores en Postman o tus dudas en el foro del curso.



Próxima sesión

Semana 14: consumo de servicios web con Angular, autenticación y autorización.